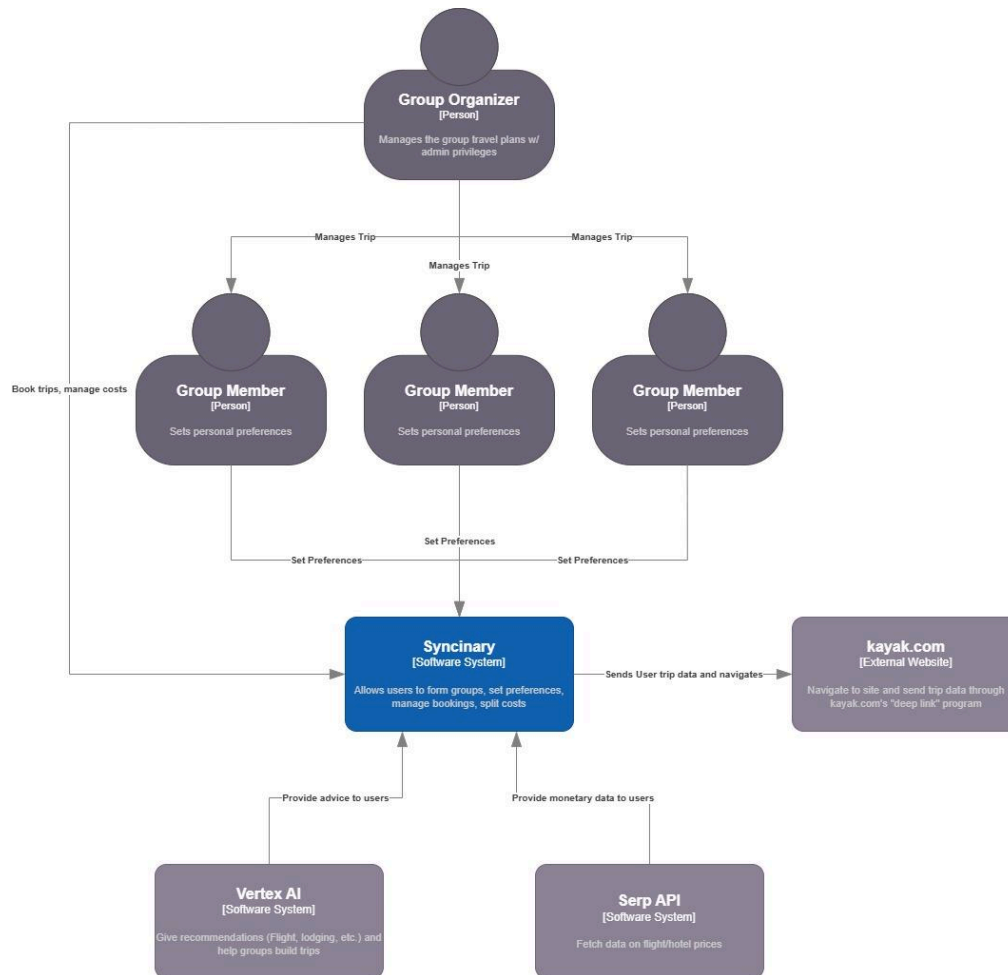
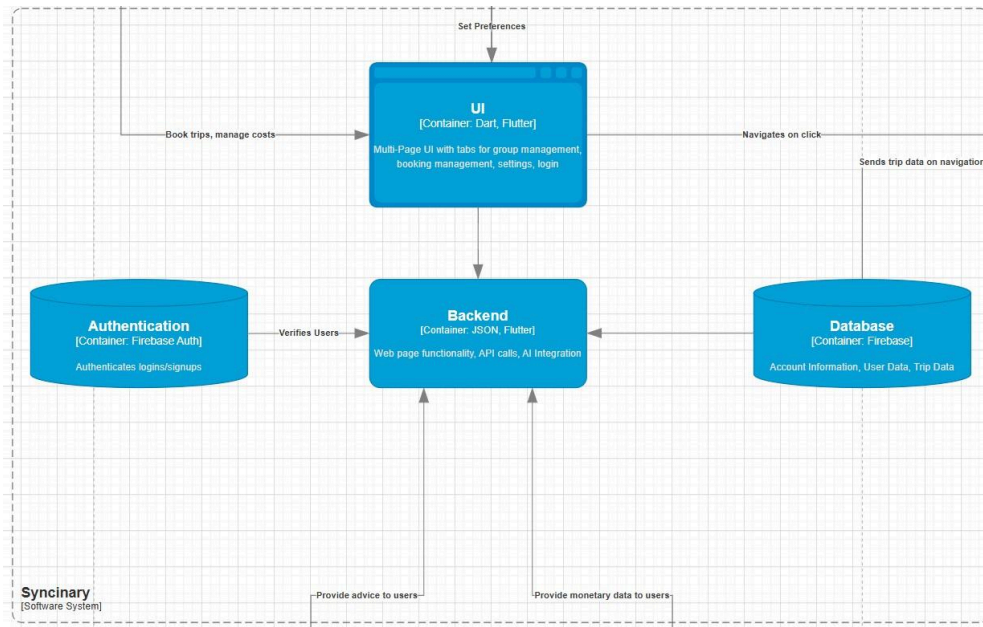


Design Diagrams

System Context Diagram:



Container Diagram:



Tech Stack

Flutter & Dart

- The Dart Framework we will be using to build our application
- Site is constructed with "widgets"
- Simple API integration



Dart is a C-like language

Firestore

- Google's noSQL database
- Integrates well with Firebase
- Wide flexibility for both storage and site hosting



Firebase Authentication

- Google's authentication
- Allows for third party login
- Highly affordable and scalable



Figma

Link to Figma:

<https://www.figma.com/design/p0MRGd1aPA6UFSMdVwjH2c/User-Flows?node-id=417-1047&t=RCihph1cOleBik86-1>

Design Trade

Jack

I considered the group management system. This consists of forming groups, setting roles, removing members and initializing the travel schedule. Design 1 was a highly modular system

that allowed users to create custom roles and preferences and have the option to join multiple groups. Design 2 was a limited system. This option restricted user freedom with pre-defined roles and no ability to add preferences. The third option was a hybrid system where roles are static but a limited number of preferences can be added.

Design	User Simplicity	Group Size	Data Structure	Preference Options	Role Options	Score
001	3	10	3	10	10	6.5
002	10	4	8	5	6	6.9
003	8	4	8	9	6	7.7

Ultimately, I selected design 3. The main tradeoffs were simplicity and stability vs user choice. With every option we allow the users to customize the risk of errors increases. For example, option 1 allowed for near total freedom in adding preference options and roles. With this, there was guaranteed to be issues with the UI and the AI integration. The second option was almost too restrictive by forcing users into predefined settings. The third option takes the middle road allowing some freedom with preferences but capping the maximum number of preferences.

Prisha

The design trade study I was responsible for was an analysis on the LLM-Assisted Recommendation system. Below are the three different designs we considered for our website:

D1 – LLM + Rule-Based Filtering

For this first design, we would need to use an LLM to generate initial travel recommendations. This would be followed by a rule-based system that enforces strict constraints such as budget, dates, and availability.

D2 – Fully Integrated LLM + API Data

For this next design, we would have the LLM directly process both structured API data and user inputs to generate recommendations in a single step.

D3 – LLM with Caching and Batch Processing

For this final design, we would rely on periodically cached API data for the LLM to use to generate recommendations.

1 = Very Poor

5 = Average

10 = Excellent

Design Alternative	Recommendation Relevance (30%)	Response Latency (20%)	Cost Efficiency (20%)	Data Integregation (20%)	Explainability (10%)	Total Weighted Score
D1 – LLM + Rule-Based Filtering	7/10	9/10	9/10	8/10	9/10	8.2
D2 – Fully Integrated LLM + API Data	9/10	5/10	5/10	7/10	5/10	6.6
D3 – LLM + Caching & Batch Processing	7/10	8/10	10/10	6/10	7/10	7.6

Summary of why D1 was chosen:

Based on the weighted evaluation criteria, D1 is the most suitable design. It achieves the highest overall score due to how it balances recommendation relevance, low latency, and cost efficiency. While D2 offers higher personalization, its increased cost and latency make it impractical for a resource-constrained student project. D3 improves scalability and cost but introduces potential issues with recommendation relevance due to using stale data. This is critical in travel planning so it's not good to compromise on that. Therefore, D1 provides the best trade-off between performance, reliability, and feasibility for our system.

Shubhi

The subsystem that I selected is the authentication system.

ID: AUTH-1 - Firebase Authentication System

Users would create accounts, log in, and log out through Firebase so Firebase would manage the entire authentication process. Once a user is authenticated, their identity is used to access their stored data in Firebase.

ID: AUTH-2 - External OAuth Authentication System

This design relies on external providers such as Google or Apple for authentication. Users sign in through those providers and the system then receives a verified identity token which is used to map the user to an internal account which connects them to their trip information.

ID: AUTH-3 - Custom Authentication System

This design builds a custom authentication system within Syncinary. The team would handle account creation, password storage, login verification, and session management directly.

Design Alternatives	Authentication security	Integration with existing tech stack	Usability	Reliability	Easy Implementation	Total Weighted Score
AUTH-1	9	10	8	9	9	9
AUTH-2	8	8	9	8	7	8.1
AUTH-3	6	5	6	6	4	5.6

Based on the trade study matrix above, AUTH-1: Firebase Authentication is the best design for the system. It has the highest total weighted score and performs well across all the 5 criteria especially in security, integration with existing tech stack, reliability, and easy implementation. This design fits directly with the current use of Firebase database and reduces the amount of custom implementation needed for integration. AUTH-2 does provide better usability but it would be more difficult to integrate with existing tech stack. AUTH-3 offers more control but has lower scores because it is extremely difficult to implement and not feasible for the duration of the project. Therefore, AUTH-1 is the best option because the only tradeoff is usability for better authentication security, integration with existing tech stack, reliability, and easy implementation.

Ethan

The subsystem I selected is the API integration subsystem.

Design A: Centralized Backend Aggregator

This design uses a backend service to call external travel APIs, normalize the data, and cache results. It allows users to view and compare structured results directly within the application before booking.

Design B: Direct External Linking

This design redirects users to external booking platforms using search URLs and relies on those platforms for results. The application does minimal processing, making it simple and fast to implement.

Design C: Multi-Provider Service Layer

This design integrates multiple travel API providers and combines their results into one output with fallback options. It improves reliability and coverage but introduces more complexity in handling and maintaining the data.

Criterion	Weight	Design A	Weighted	Design B	Weighted	Design C	Weighted
Response Time	25	8	2.00	8	2.00	6	1.50
Reliability	25	8	2.00	5	1.25	9	2.25
Implementation Complexity	15	6	0.90	9	1.35		

Based on the trade study matrix, Design A is the best choice. It has the highest overall performance across the most important criteria, especially in response time and reliability, while still maintaining a reasonable implementation complexity. Although Design B scores higher in implementation simplicity, it sacrifices reliability, which is critical for this subsystem. Design C performs well in reliability but has too much complexity and weaker response time. Therefore, Design A provides the best balance between performance and practicality, making it the most effective option.